

Git/Gitlab

Adolfo De Unanúe y Fernanda Sobrino

09/28/2021

Intro

Basic Git

Revert

Collaborative Git

Intro

Problema

- ▶ Digamos que tu y tu compañero de trabaja están actualizando código en el mismo proyecto
- ▶ Tu haces algunos cambios, los guardas y los vuelves a subir
- ▶ Esto va a funcionar pero qué pasa si tu compañero esta trabajando en el mismo código que tu al mismo tiempo?
- ▶ Uno de ustedes estas a punto a de sorbe escribir y borrar su trabajo

Solución: Version control

- ▶ Guarda diferentes archivos o *versiones* a través de las distintas etapas del proyecto
- ▶ Puedes anotar los cambios en cada *versión*
- ▶ Fácilmente puedes regresar a versiones anteriores
- ▶ Muy útil para trabajar en equipo
- ▶ Facilita la resolución de errores

Basic Git

Git

- ▶ Versión control software
- ▶ Guarda cada versión como un snapshot
- ▶ La copia de trabajo de código de cada desarrollador también es un repositorio que puede contener el historial completo de cambios

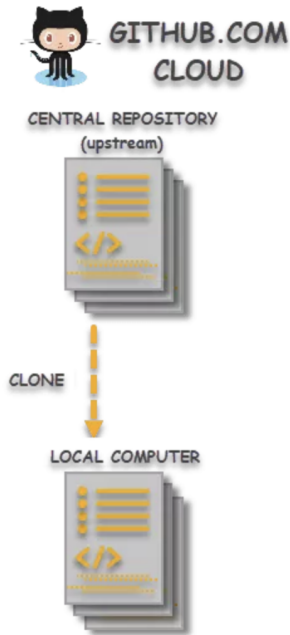
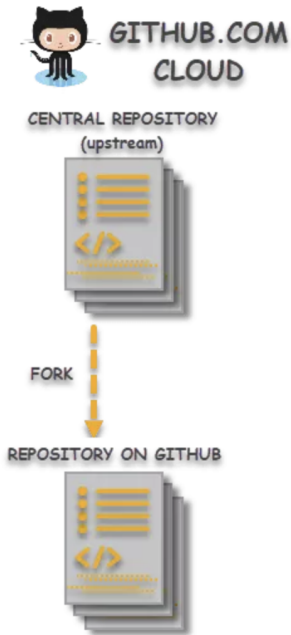
Algunas definiciones

- ▶ **repository:** un directorio o espacio de almacenamiento donde pueden vivir tus proyectos
- ▶ **commit:** toma un snapshot del tu repositorio al cual puedes evaluar o restaurar el proyecto a cualquier estado anterior
- ▶ **push:** empuja los cambios desde tu máquina a la nube
- ▶ **pull:** extrae las cosas de la nube a tu máquina, simplemente extrae los nuevos cambios que se han realizado desde que extrajiste el repositorio la última vez

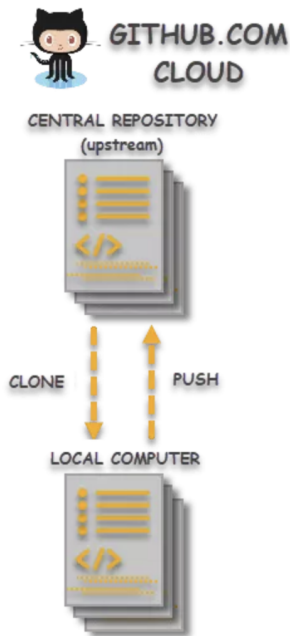
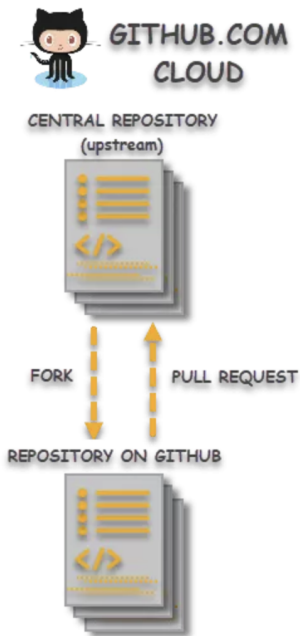
Algunas definiciones

- ▶ **clone:** crea una copia local del repositorio
- ▶ **fork:** crea una copia de un repositorio publico para después ser clonado
- ▶ **pull request:** no un comando real sino una función integrada en git. Su compañero de trabajo le solicita a usted que introduzca sus cambios en su repositorio

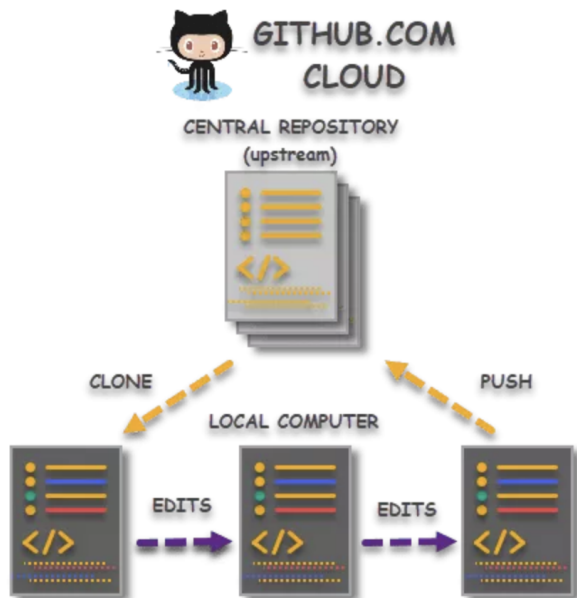
Fork vs Clone



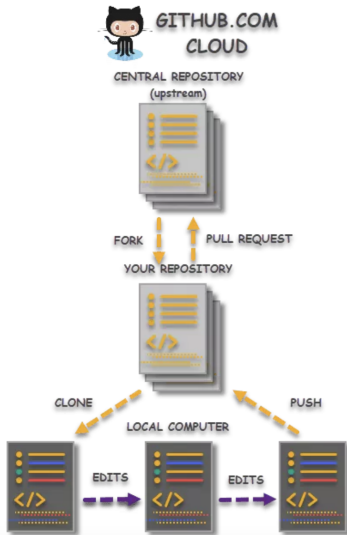
Fork vs Clone



Cloning



Forking



Cómo clono el repo de mi equipo a mi máquina?

1. En este [link](#) busco mi equipo, por ejemplo Equipo A
2. Abro el repo de mi equipo y copio el link para clonar el repo a mi máquina
 - a. si quieren usar clone with ssh, tienen que darle a gitlab su ssh (en su usuario van a Edit profile/SSH Keys y ahí la ponen)
 - b. si usan el clone with HTTPS les va a pedir su usuario y su contraseña de gitlab
3. En su terminal van al lugar donde van a querer la carpeta `cd donde/quiero/el/repo/de/mi/equipo/`
4. Ya en ese lugar: `git clone el_link_que_copiaron`, esto crea una carpeta de su equipo en su máquina

Cómo agrego un archivo a esta carpeta? Vamos a crear un readme file

1. Asegure de estar en la carpeta de su equipo `cd /donde/sea/que/este/EquipoA`
2. `git switch -c main` crea una rama llamada main y se asegura que estemos en ella
3. `touch README.md` crea un archivo vacío que se llama así o actualiza el timestamp del archivo si es que existe
4. `git add README.md` agrega el archivo que creamos a nuestro folder (en nuestra máquina)
5. `git commit -m "add README"` toma el snapshot de nuestro repo para incluir el nuevo archivo. `git commit` hace el commit, `-m "lo que sea"` agrega un comentario a este commit
6. `git push -u origin main` lo manda al repo que esta en gitlab

Tips

- ▶ Usen `git status` para saber en que etapa están sus cambios en el commit
- ▶ Lo anterior no es 'colaborativo' per se, no soluciona el problema descrito en las primeras slides
- ▶ Por ahora gitlab funciona solo como una carpeta donde todos pueden cambiar y subir cosas pero si dos personas hacen `git push` al mismo documento alguien pierde su trabajo

Revert

revert

- ▶ es como un 'Undo' para git
- ▶ pero no es uno tradicional
- ▶ en vez de quitar el commit de la historia del proyecto, descubre como invertir los cambios introducidos y agrega uno con el contenido inverso
- ▶ Prevee que git pierda historia

Cómo se usa?

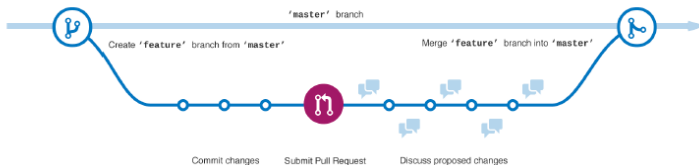
- ▶ **OJO:** cambios que no haz salvado no van a aparecer en ningún lado, si tienes cambios sin hacer commit git te va a preguntar si quieres hacer commit
- ▶ **OJO:** revert no te va a llevar atras a un estado en especifico, solo va a deshacer los cambios de tu ultimo commit
- ▶ `git revert`

Collaborative Git

Branches

- ▶ **branching:** forma de trabajar en diferentes versiones de un repositorio a la vez
 - ▶ por default tu repositorio tiene una `main` branch
 - ▶ usa branches para experimentar y hacer ediciones antes de darle commit en la rama principal
 - ▶ una nueva rama es una copia de `main`
 - ▶ si tu compañero cambia la `main` mientras tu estas trabajando en tu rama puedes hacer `git pull` para actualizar tu rama

Branches



- ▶ `main` branch
- ▶ nueva rama llamada `feature`
- ▶ el viaje que `feature` hace antes de unirse a `main`

Branches

Las ramas te previenen de tener cosas como estas:

- ▶ `story.Rmd`
- ▶ `story-joe-edit.Rmd`
- ▶ `story-joe-edit-reviewed.Rmd`

How to create a new branch?

- ▶ Existen varias maneras de crear una nueva rama: `git checkout -b my-new-branch`
- ▶ Una vez que creas tu rama, puedes hacer lo mismo que antes `git add .` y `git commit -m 'mi nueva rama o algo'`
- ▶ Haces el push a gitlab `git push origin my-new-branch`
- ▶ gitlab te va a mandar un link directo para hacer la merge request
- ▶ copia el link y pegalo en tu browser
- ▶ ve a el gitlab de tu equipo y entre todos resuelvan si hay algún merge issue